



浙江大學
ZHEJIANG UNIVERSITY

Logic and Computer Design Fundamentals

Chapter 2 –Combinational Logic Circuits(2)

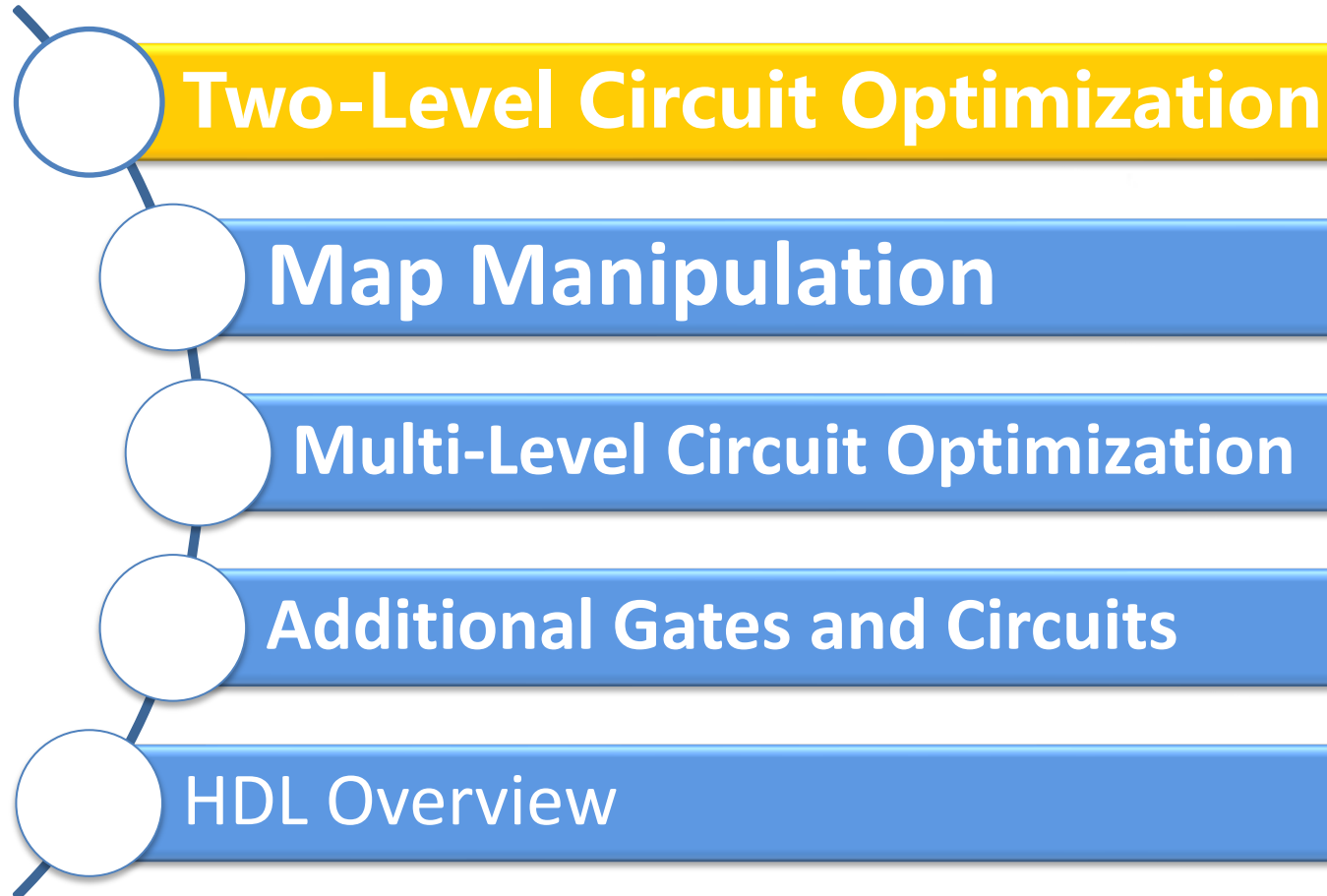
Ma De (马德)

made@zju.edu.cn

2023

College of Computer Science, Zhejiang University

Course Outline



Circuit Optimization

- ❑ **Goal: To obtain the simplest implementation for a given function**
- ❑ **Optimization is a more formal approach to simplification that is performed using a specific procedure or algorithm**
- ❑ **Optimization requires a cost criterion to measure the simplicity of a circuit**
- ❑ **Distinct cost criteria we will use:**
 - **Literal cost (L)**
 - **Gate input cost (G)**
 - **Gate input cost with NOTs (GN)**

Literal Cost

- Literal – a variable or its complement
- Literal cost – the number of literal appearances in a Boolean expression corresponding to the logic circuit diagram
- Examples:

$$\blacksquare F = BD + A\bar{B}C + A\bar{C}\bar{D} \quad L = 8$$

$$\blacksquare F = BD + A\bar{B}C + A\bar{B}\bar{D} + AB\bar{C} \quad L = 11$$

$$\blacksquare F = (A + B)(A + D)(B + C + \bar{D})(\bar{B} + \bar{C} + D) \quad L = 10$$

■ Which solution is best?

The first solution is best

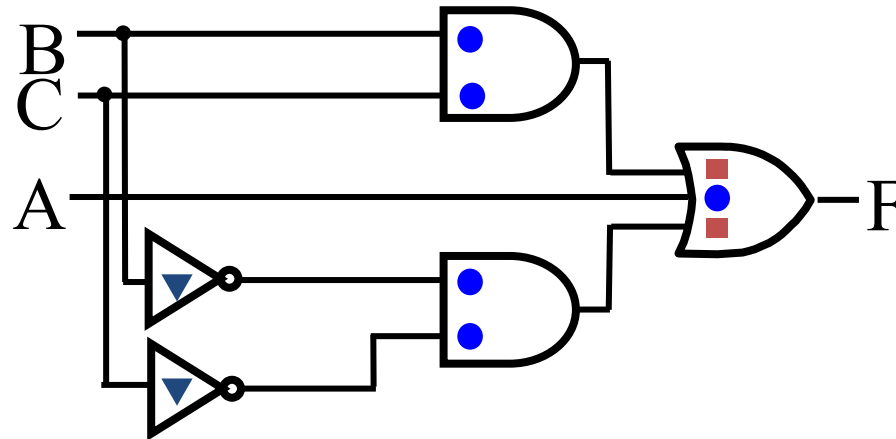
Gate Input Cost

- Gate input costs - the number of inputs to the gates in the implementation corresponding exactly to the given equation or equations. (G - inverters not counted, GN - inverters counted)
- For SOP and POS equations, it can be obtained from the equation(s) by finding the sum of:
 - all literal appearances
 - the number of terms excluding single literal terms, (G) and
 - optionally, the number of distinct complemented single literals (GN).
- Example:
 - $F = BD + \overline{A}\overline{B}C + A\overline{C}\overline{D}$ $G = 11, GN = 14$
 - $F = BD + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{D} + ABC\overline{C}$ $G = 15, GN = 18$
 - $F = (A + B)(A + D)(B + C + \overline{D})(\overline{B} + \overline{C} + D)$ $G = 14, GN = 17$
 - Which solution is best?

The 1st solution is best

Cost Criteria (continued)

- Example 1:
- $F = A + B \cdot C + \overline{B} \cdot \overline{C}$
- $GN = G + 2 = 9$
- $L = 5$
- $G = L + 2 = 7$



- L (literal count) counts the AND inputs and the single literal OR input.
- G (gate input count) adds the remaining OR gate inputs
- GN (gate input count with NOTs) adds the inverter inputs

Cost Criteria (continued)

□ Example 2:

□ $F = A B C + \bar{A} \bar{B} \bar{C}$

□ $L = 6 \quad G = 8 \quad GN = 11$

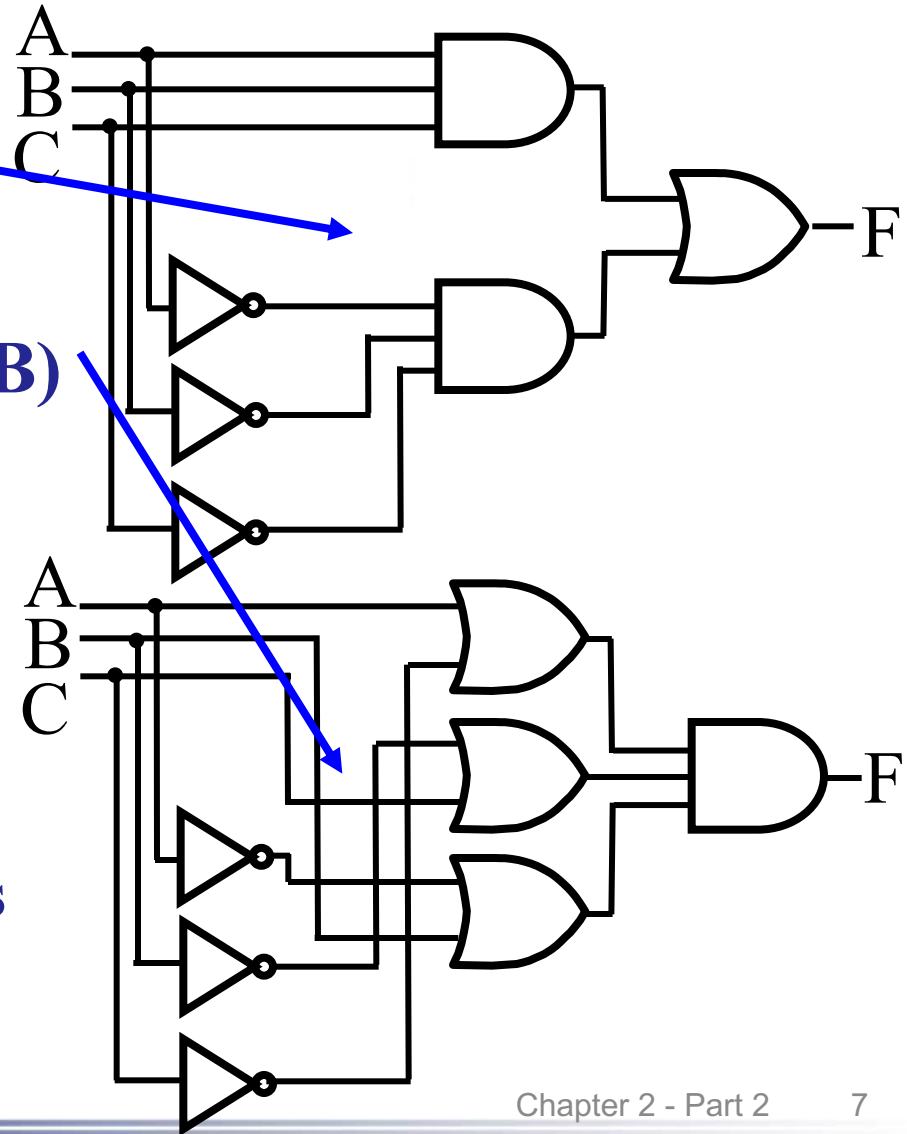
□ $F = (A + \bar{C})(\bar{B} + C)(\bar{A} + B)$

□ $L = 6 \quad G = 9 \quad GN = 12$

□ Same function and same literal cost

□ But first circuit has better gate input count and better gate input count with NOTs

□ Select it!

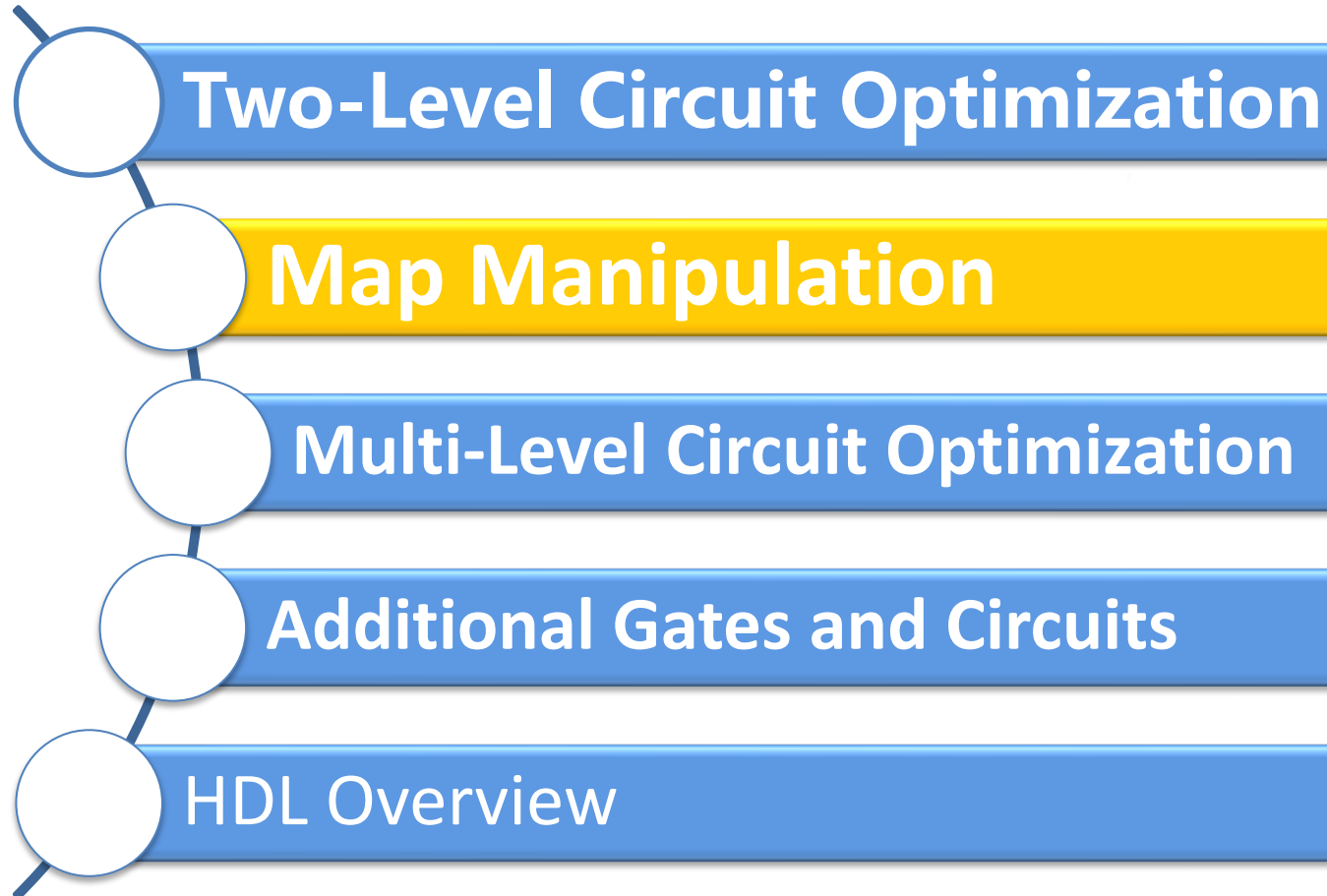




Boolean Function Optimization

- ❑ Minimizing the gate input (or literal) cost of a (a set of) Boolean equation(s) reduces circuit cost.
- ❑ We choose gate input cost.
- ❑ Boolean Algebra and graphical techniques are tools to minimize cost criteria values.
- ❑ Some important questions:
 - When do we stop trying to reduce the cost?
 - Do we know when we have a minimum cost?
- ❑ Treat optimum or near-optimum cost functions for two-level (SOP and POS) circuits first.
- ❑ Introduce a graphical technique using Karnaugh maps (K-maps, for short)

Course Outline



Karnaugh Maps (K-map)

- A K-map is a collection of squares
 - Each square represents a minterm
 - The collection of squares is a graphical representation of a Boolean function
 - Adjacent squares differ in the value of one variable
 - Alternative algebraic expressions for the same function are derived by recognizing patterns of squares
- The K-map can be viewed as
 - A reorganized version of the truth table
 - A topologically-warped Venn diagram as used to visualize sets in algebra of sets

Two Variable Maps

□ A 2-variable Karnaugh Map:

- Note that minterm m_0 and minterm m_1 are “adjacent” and differ in the value of the variable y
- Similarly, minterm m_0 and minterm m_2 differ in the x variable.
- Also, m_1 and m_3 differ in the x variable as well.
- Finally, m_2 and m_3 differ in the value of the variable y

	$y = 0$	$y = 1$
$x = 0$	$m_0 =$ $\underline{\quad}\underline{\quad}$ $x \ y$	$m_1 =$ $\underline{\quad}\underline{\quad}$ $x \ y$
$x = 1$	$m_2 =$ $\underline{\quad}\underline{\quad}$ $x \ y$	$m_3 =$ $\underline{\quad}\underline{\quad}$ $x \ y$

K-Map and Truth Tables

- The K-Map is just a different form of the truth table.
- Example – Two variable function:
 - We choose a,b,c and d from the set $\{0,1\}$ to implement a particular function, $F(x,y)$.

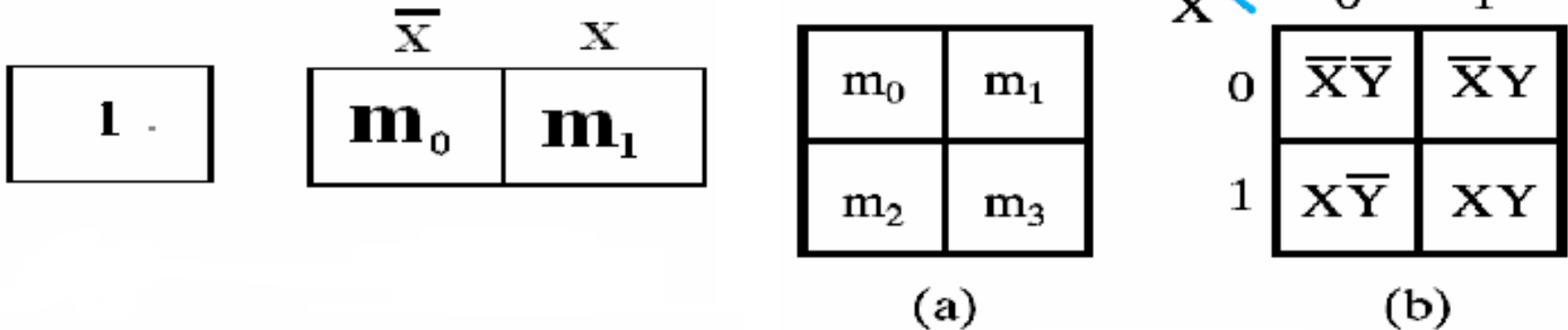
Function Table

Input Values (x,y)	Function Value $F(x,y)$
0 0	a
0 1	b
1 0	c
1 1	d

K-Map

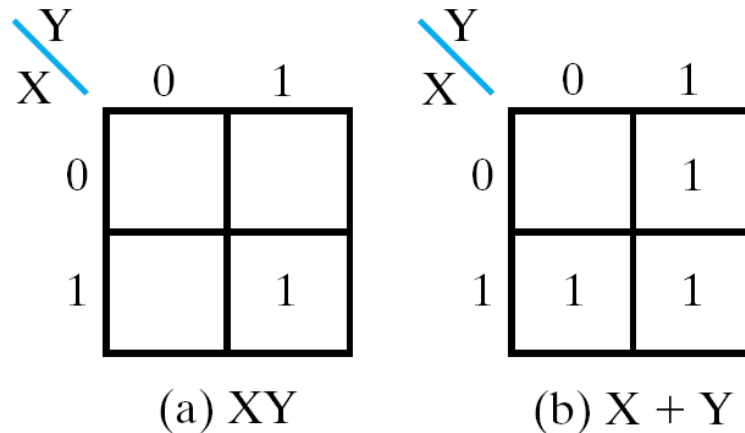
	$y = 0$	$y = 1$
$x = 0$	a	b
$x = 1$	c	d

Examples



Const, one variable, two variable K-maps

$$F(X, Y) = m_1 + m_2 + m_3 = \overline{X}Y + X\overline{Y} + XY = X + Y$$



Present function with K-maps

Three variable maps

Has 2^3 minterms, and 8 squares.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

a

		YZ		Y	
		00	01	11	10
X	0	$\bar{X}\bar{Y}\bar{Z}$	$\bar{X}\bar{Y}Z$	$\bar{X}YZ$	$\bar{X}Y\bar{Z}$
X	1	$X\bar{Y}\bar{Z}$	$X\bar{Y}Z$	XYZ	$XY\bar{Z}$
		Z			

b

	$\bar{y}$	y		
$\bar{x}$	0	1	3	2
x	4	5	7	6
	$\bar{z}$	z	$\bar{z}$	

	1	1	

Example:

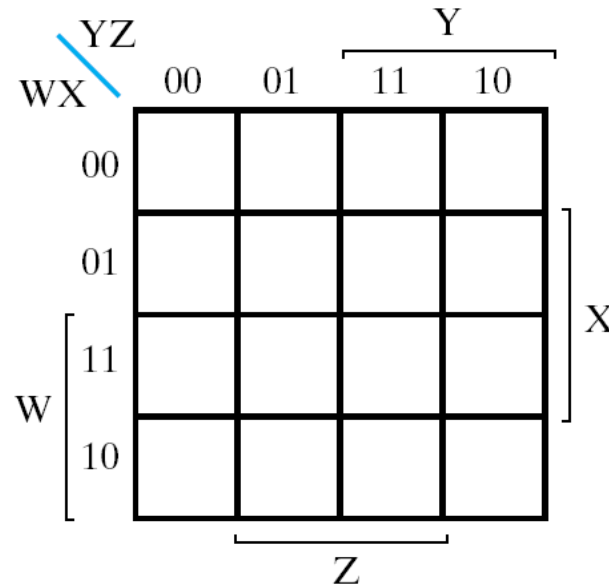
$$F(X, Y, Z) = m_5 + m_7 = X\bar{Y}Z + XYZ = XZ(\bar{Y} + Y) = XZ$$

Four variable maps

Has 2^4 minterms, and 16 squares.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)



(b)

	YZ 00	YZ 01	YZ 11	YZ 10
WX 00	0	1	3	2
WX 01	4	5	7	6
WX 11	12	13	15	14
WX 10	8	9	11	10

Five variable maps



Has 2^5 minterms, and 32 squares.

XYZ VW		000	001	011	010	110	111	101	100
00		0	1	3	2	6	7	5	4
01		8	9	11	10	14	15	13	12
11		24	25	27	26	30	31	29	28
10		16	17	19	18	22	23	21	20

Example: Combining Squares

□ Example: $F(x,y) = x$

$F = x$	$y = 0$	$y = 1$
$x = 0$	0	0
$x = 1$	1	1

□ For function $F(x,y)$, the two adjacent cells containing 1's can be combined using the Minimization Theorem:

$$F(x, y) = x\bar{y} + xy = x$$

Example: Combining Squares

□ Example: Let $F = \Sigma m(2,3,6,7)$

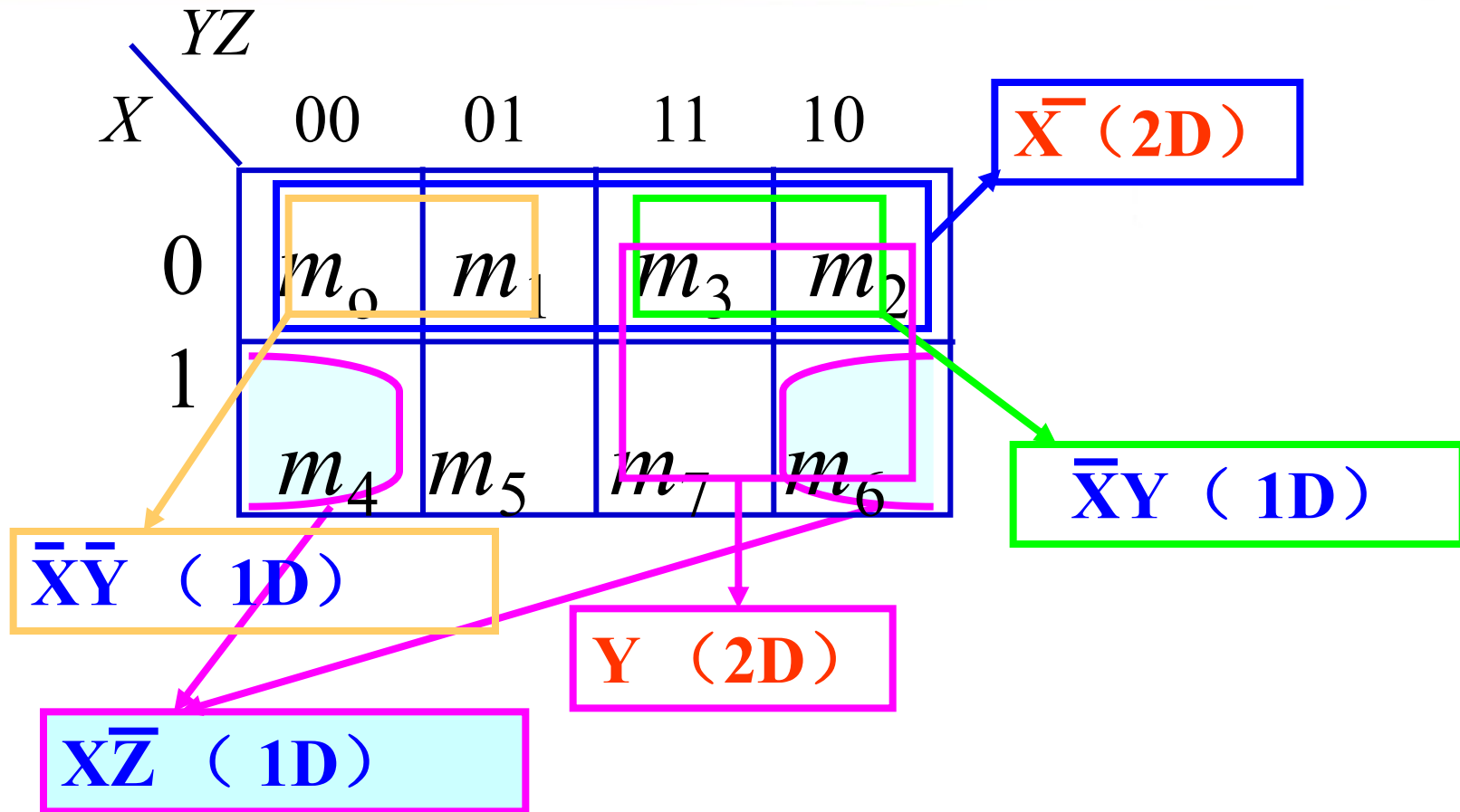
			y	
	0	1	3 1	2 1
x	4	5	7 1	6 1
			z	

□ Applying the Minimization Theorem three times:

$$\begin{aligned}
 F(x, y, z) &= \bar{x} y z + x y z + \bar{x} y \bar{z} + x y \bar{z} \\
 &= yz + y\bar{z} \\
 &= y
 \end{aligned}$$

□ Thus the four terms that form a 2×2 square correspond to the term "y".

Dimension of Squares



One variable can be removed with 1D square, and two variables can be removed with 2D square

Example: Combining Squares

X \ Y	0	1
	0	1
0		
1	1	1

X \ Y	0	1
	0	1
0	1	
1	1	

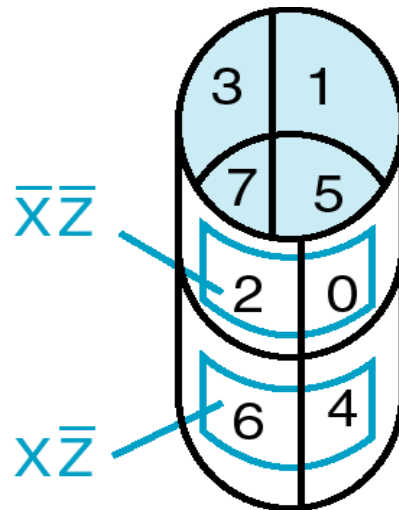
X \ Y	0	1
	0	1
0		1
1	1	1

Combining Squares in two variable maps

Example: Combining Squares

$X \backslash YZ$	00	01	11	10
0	1			1
1	1			1

$X \backslash YZ$	00	01	11	10
0		1	1	
1		1	1	



$X \backslash YZ$	00	01	11	10
0	1	1	1	1
1	1	1	1	1

Combining Squares in three variable maps

Example: Combining Squares

$XY \backslash ZW$	00	01	11	10
000	1			1
111		1	1	
10		1	1	
	1			1

$XY \backslash ZW$	00	01	11	10
000		1	1	
111		1	1	
10	1	1	1	1
		1	1	

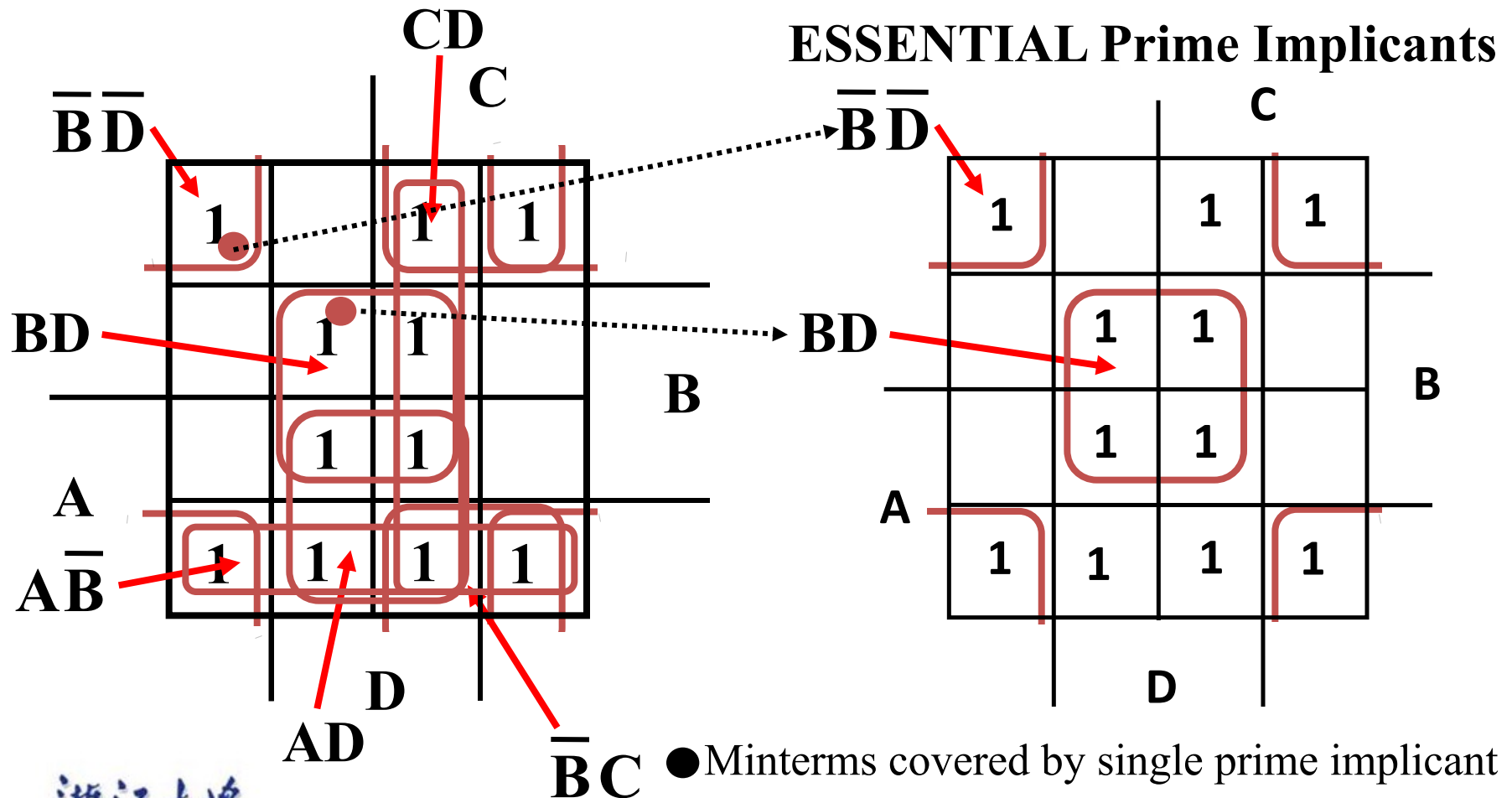
$XY \backslash ZW$	00	01	11	10
000		1	1	
111	1			1
10	1			1
		1	1	

Systematic Simplification

- ❑ A Prime Implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map into a rectangle with the number of squares a power of 2.
- ❑ A prime implicant is called an Essential Prime Implicant if it is the only prime implicant that covers (includes) one or more minterms.
- ❑ Prime Implicants and Essential Prime Implicants can be determined by inspection of a K-Map.
- ❑ A set of prime implicants "covers all minterms" if, for each minterm of the function, at least one prime implicant in the set of prime implicants includes the minterm.

Example of Prime Implicants

Find ALL Prime Implicants





Optimization Algorithm

- ❑ Find all prime implicants.
- ❑ Include all essential prime implicants in the solution
- ❑ Select a minimum cost set of non-essential prime implicants to cover all minterms not yet covered:
 - Obtaining an optimum solution: See Reading Supplement - More on Optimization
 - Obtaining a good simplified solution: Use the Selection Rule



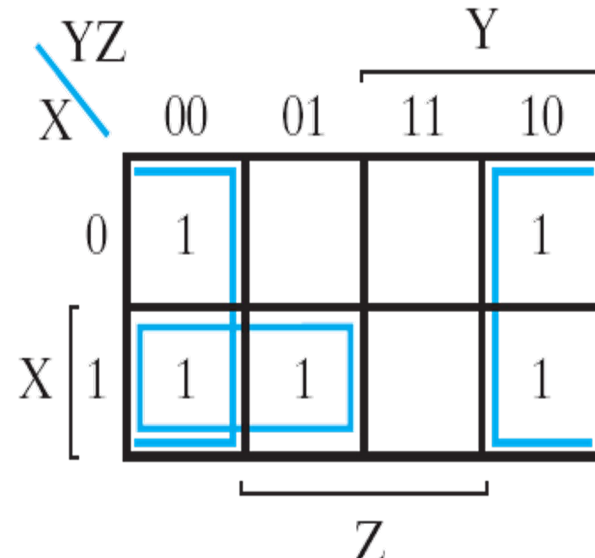
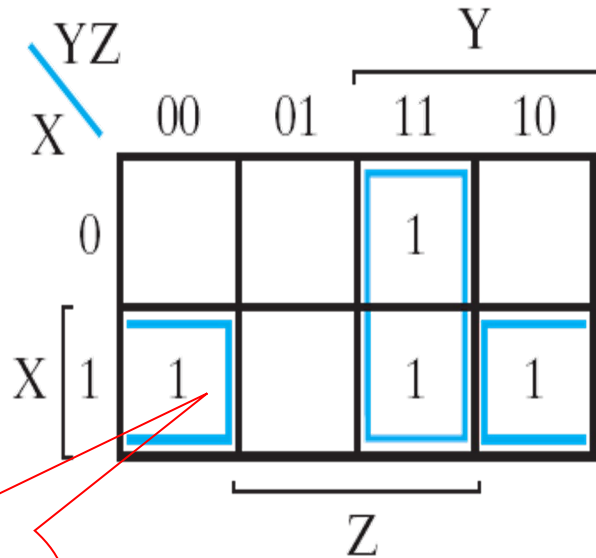
Prime Implicant Selection Rule

- Minimize the overlap among prime implicants as much as possible. In particular, in the final solution, make sure that each prime implicant selected includes at least one minterm not included in any other prime implicant selected.

K-Map Simplification

$$F_1(X, Y, Z) = m_3 + m_4 + m_6 + m_7 = \sum m(3, 4, 6, 7)$$

$$F_2(X, Y, Z) = m_0 + m_2 + m_4 + m_5 + m_6 = \sum m(0, 2, 4, 5, 6)$$



Prime
Implicants

$$\begin{aligned} (a) F_1(X, Y, Z) &= \sum m(3, 4, 6, 7) \\ &= YZ + X\bar{Z} \end{aligned}$$

$$\begin{aligned} (b) F_2(X, Y, Z) &= \sum m(0, 2, 4, 5, 6) \\ &= \bar{Z} + X\bar{Y} \end{aligned}$$

K-Map Simplification

$$F_1 = F(X, Y, Z) = \sum m(1, 3, 4, 5, 6)$$

$$F_2 = F(X, Y, Z) = \sum m(1, 2, 3, 5, 7)$$

		Y			
		YZ			
		00	01	11	10
X	0		1	1	
	1	1	1		1

$$\begin{aligned}
 F(X, Y, Z) &= \sum m(1, 3, 4, 5, 6) \\
 &= \bar{X}Z + X\bar{Y} + X\bar{Z} \\
 &= \bar{X}Z + \bar{Y}Z + X\bar{Z}
 \end{aligned}$$

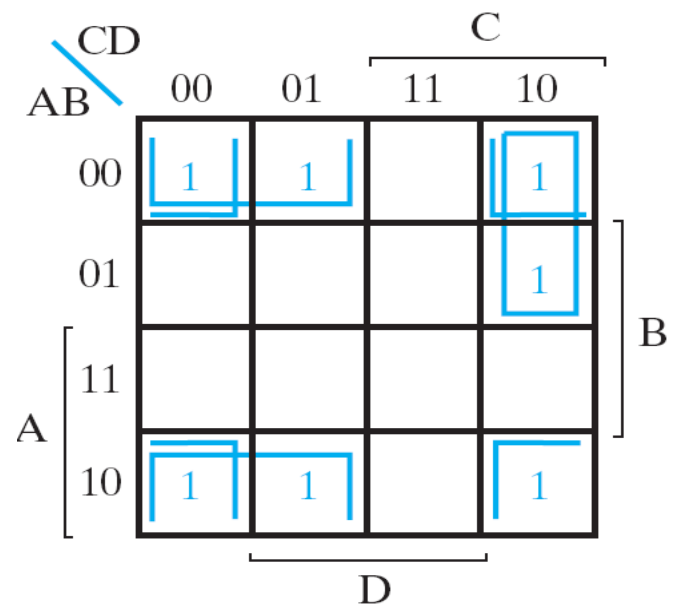
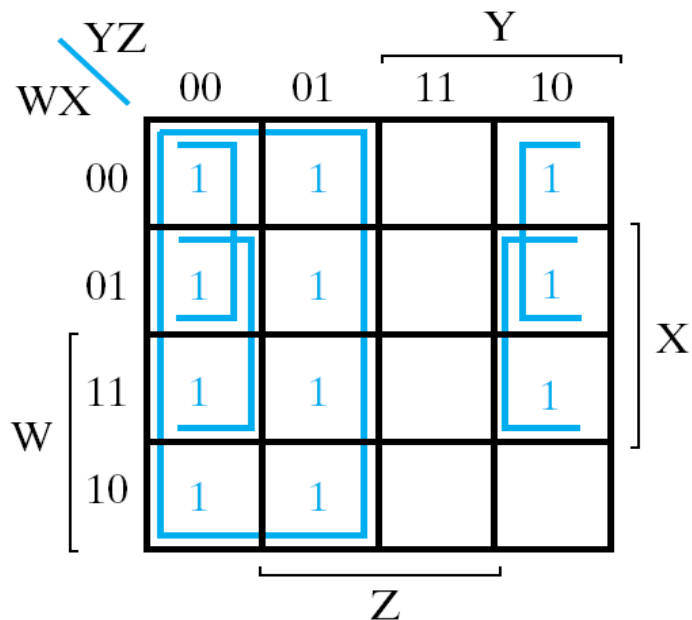
		Y			
		YZ			
		00	01	11	10
X	0		1	1	1
	1		1	1	

$$\begin{aligned}
 F(X, Y, Z) &= \sum m(1, 2, 3, 5, 7) \\
 &= Z + \bar{X}Y
 \end{aligned}$$

K-Map Simplification

$$F(W, X, Y, Z) = \sum m(0,1,2,4,5,6,8,9,12,13,14)$$

$$F(A, B, C, D) = \overline{A}\overline{B}\overline{C} + \overline{B}C\overline{D} + A\overline{B}\overline{C} + \overline{A}BC\overline{D}$$

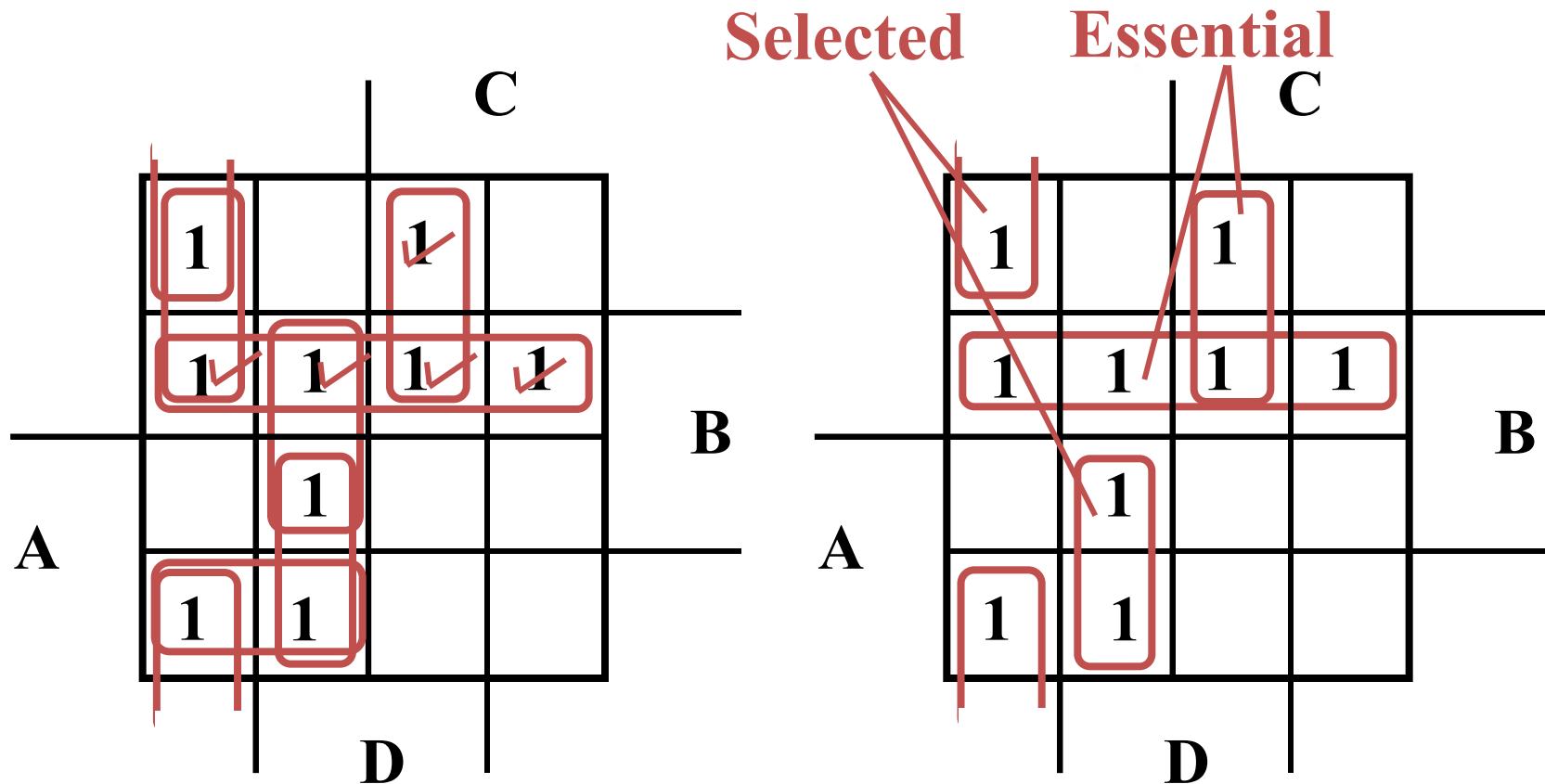


$$F(W, X, Y, Z) = \overline{Y} + \overline{W}\overline{Z} + X\overline{Z}$$

$$F(A, B, C, D) = \overline{B}\overline{D} + \overline{B}\overline{C} + \overline{A}C\overline{D}$$

Selection Rule Example

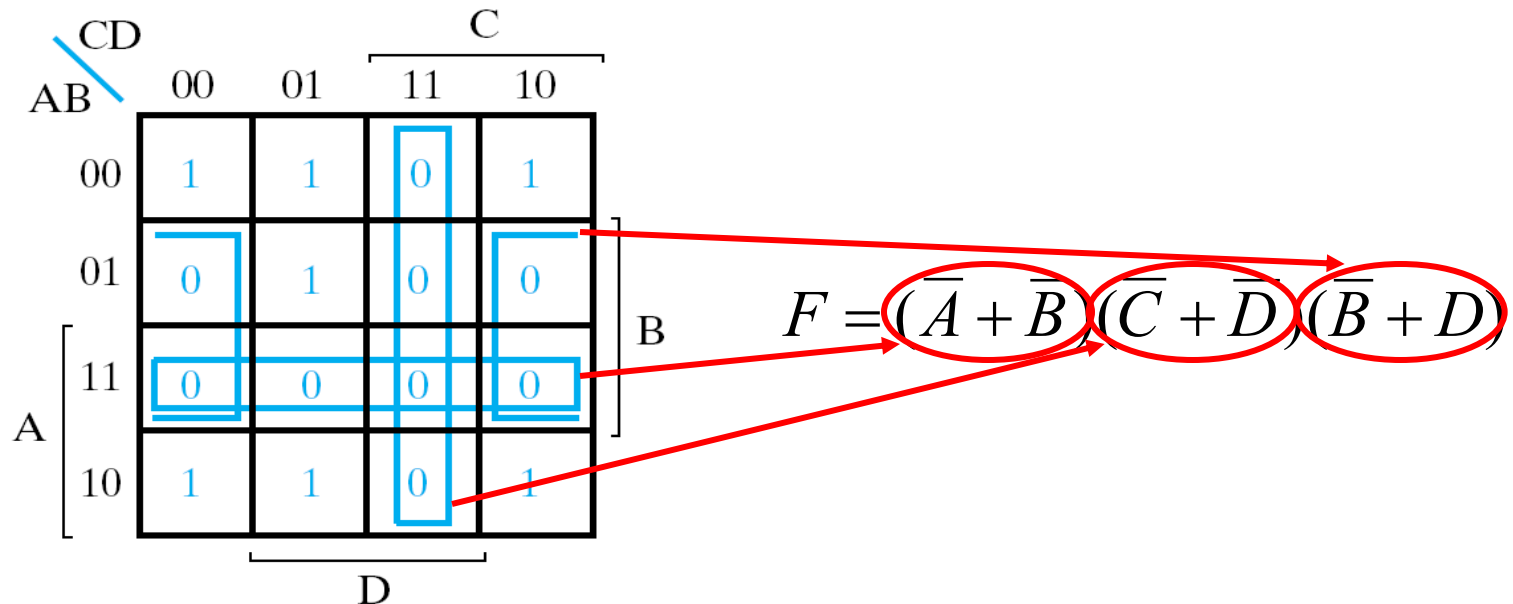
□ Simplify $F(A, B, C, D)$ given on the K-map.



✓ Minterms covered by essential prime implicants

K-Map Simplification with maxterms

Equation:
$$F(A, B, C, D) = \sum m(0, 1, 2, 5, 8, 9, 10)$$





Simplification with Maxterm: complemented function

Equation:
$$F = (\bar{A} + \bar{B} + C)(B + D)$$

Complemented function :
$$\bar{F} = AB\bar{C} + \bar{B}\bar{D}$$

Combine 1:
$$F = \bar{A}B + \bar{B}D + BC$$

CD \ AB	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	0	0	1	1
10	0	1	1	0

Combine 0:
$$\bar{F} = \bar{B}\bar{D} + AB\bar{C}$$

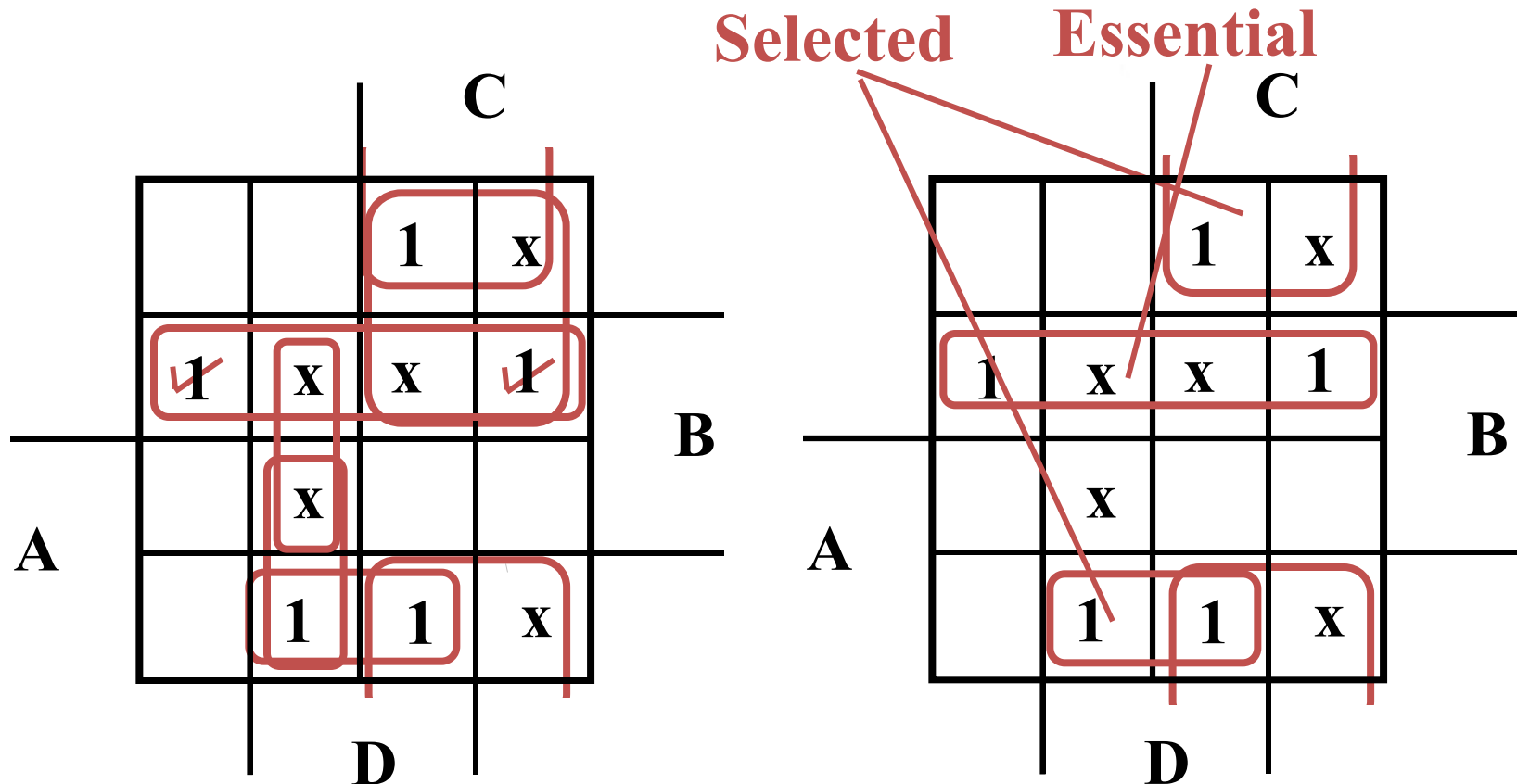
So,
$$F = (\bar{A} + \bar{B} + C)(B + D)$$

Don't Cares in K-Maps

- ❑ Sometimes a function table or map contains entries for which it is known:
 - the input values for the minterm will never occur, or
 - The output value for the minterm is not used
- ❑ In these cases, the output value need not be defined
- ❑ Instead, the output value is defined as a “don't care”
- ❑ By placing “don't cares” (an “x” entry) in the function table or map, the cost of the logic circuit may be lowered.
- ❑ Example 1: A logic function having the binary codes for the BCD digits as its inputs. Only the codes for 0 through 9 are used. The six codes, 1010 through 1111 never occur, so the output values for these codes are “x” to represent “don't cares.”

Optimization with Don't Cares

□ Simplify $F(A, B, C, D)$ given on the K-map.



✓ Minterms covered by essential prime implicants

Product of Sums Example

□ Find the optimum POS solution:

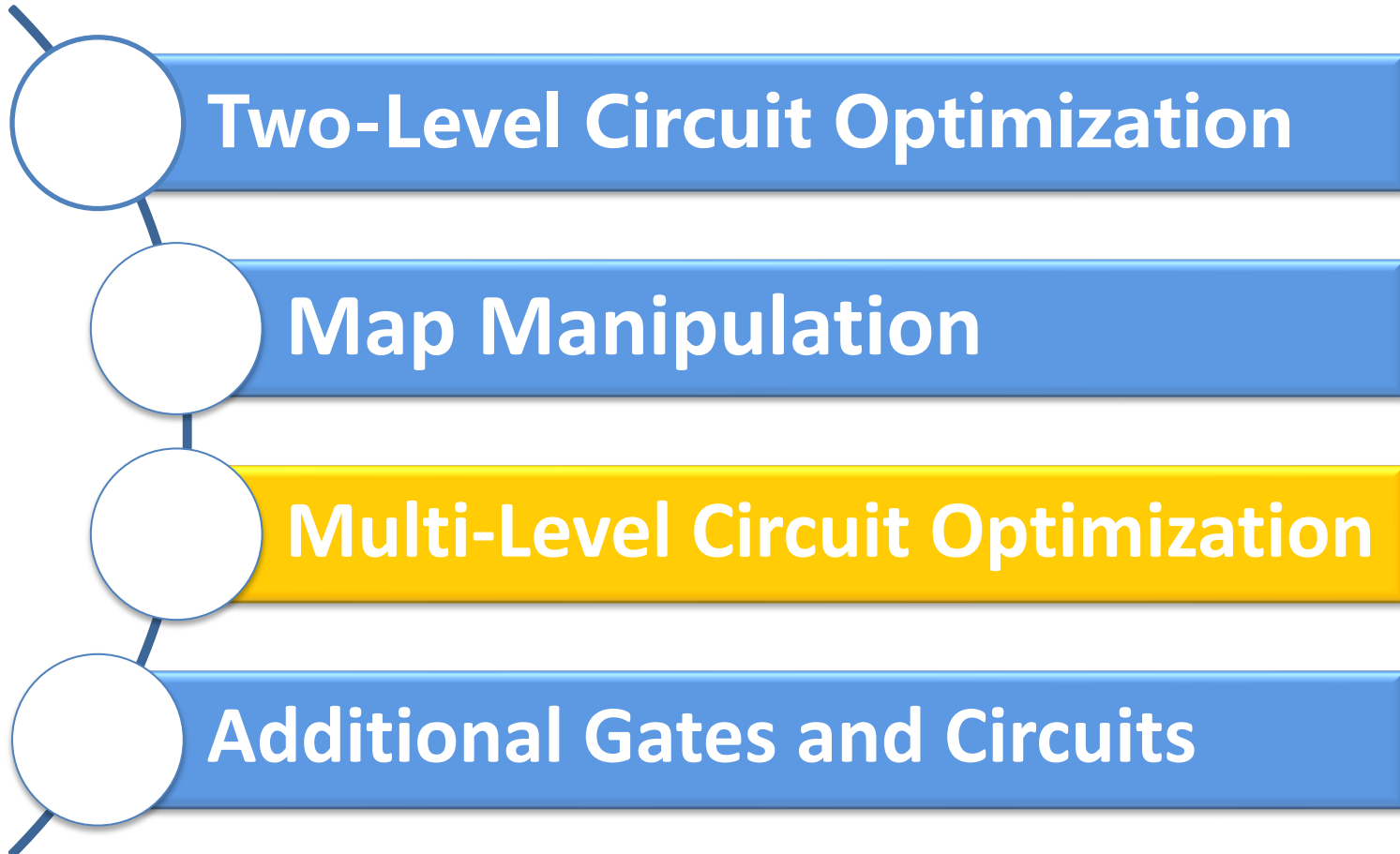
$$F(A, B, C, D) = \Sigma_m(3, 9, 11, 12, 13, 14, 15) + \Sigma_d(1, 4, 6)$$

■ Hint: Use $\bar{F}$ and complement it to get the result.

$$F' = B' D' + A' B$$

$$F = (B + D)(A + B')$$

Course Outline



Multiple-Level Optimization*

- ❑ Multiple-level circuits - circuits that are not two-level (with or without input and/or output inverters)
- ❑ Multiple-level circuits can have reduced gate input cost compared to two-level (SOP and POS) circuits
- ❑ Multiple-level optimization is performed by applying transformations to circuits represented by equations while evaluating cost

Examples

□ Algebraic Factoring

$$F = \bar{A}\bar{C}\bar{D} + \bar{A}B\bar{C} + ABC + AC\bar{D} \quad G = 16$$

■ Factoring:

$$F = \bar{A}(\bar{C}\bar{D} + B\bar{C}) + A(BC + C\bar{D}) \quad G = 18$$

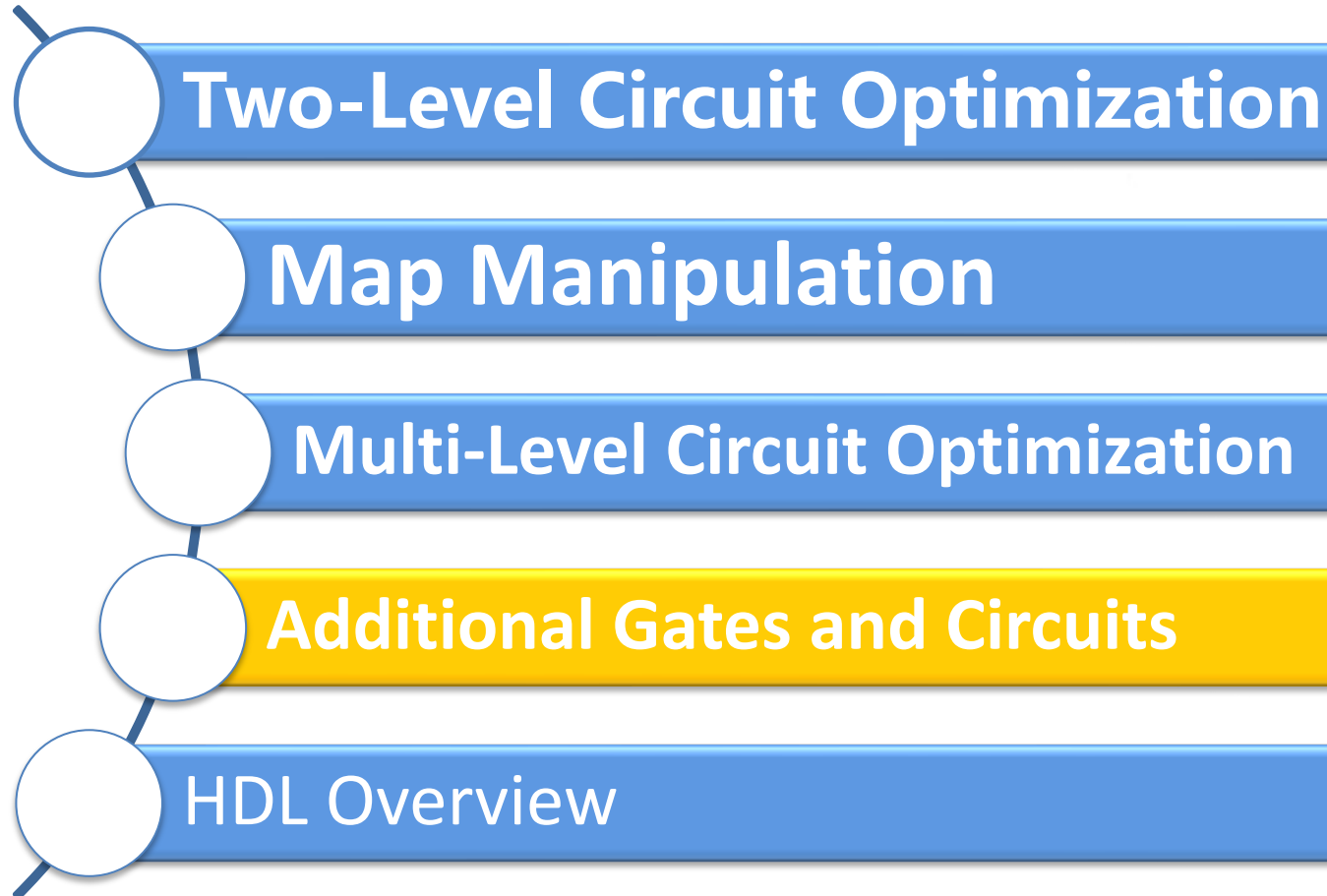
■ Factoring again:

$$F = \bar{A}\bar{C}(B + \bar{D}) + AC(B + \bar{D}) \quad G = 12$$

■ Factoring again:

$$F = (\bar{A}\bar{C} + AC)(B + \bar{D}) \quad G = 10$$

Course Outline



Other Gate Types

□ Why?

- Implementation feasibility and low cost
- Power in implementing Boolean functions
- Convenient conceptual representation

□ Gate classifications

- Primitive gate - a gate that can be described using a single primitive operation type (AND or OR) plus an optional inversion(s).
- Complex gate - a gate that requires more than one primitive operation type for its description

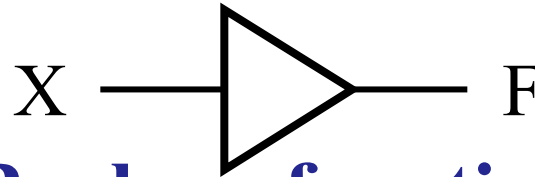
Primitive Gates

■ Symbols & truth tables for primitive gate

Name	Distinctive shape	Algebraic equation	Truth table															
AND		$F = XY$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	F	0	0	0	0	1	0	1	0	0	1	1	1
X	Y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = X + Y$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	F	0	0	0	0	1	1	1	0	1	1	1	1
X	Y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT (inverter)		$F = \overline{X}$	<table><tr><th>X</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	X	F	0	1	1	0									
X	F																	
0	1																	
1	0																	
Buffer		$F = X$	<table><tr><th>X</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	X	F	0	0	1	1									
X	F																	
0	0																	
1	1																	
3-State Buffer			<table><tr><th>E</th><th>X</th><th>F</th></tr><tr><td>0</td><td>0</td><td>Hi-Z</td></tr><tr><td>0</td><td>1</td><td>Hi-Z</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	E	X	F	0	0	Hi-Z	0	1	Hi-Z	1	0	0	1	1	1
E	X	F																
0	0	Hi-Z																
0	1	Hi-Z																
1	0	0																
1	1	1																
NAND		$F = \overline{X \cdot Y}$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	F	0	0	1	0	1	1	1	0	1	1	1	0
X	Y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{X + Y}$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	F	0	0	1	0	1	0	1	0	0	1	1	0
X	Y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																

Buffer

- A buffer is a gate with the function $F = X$:

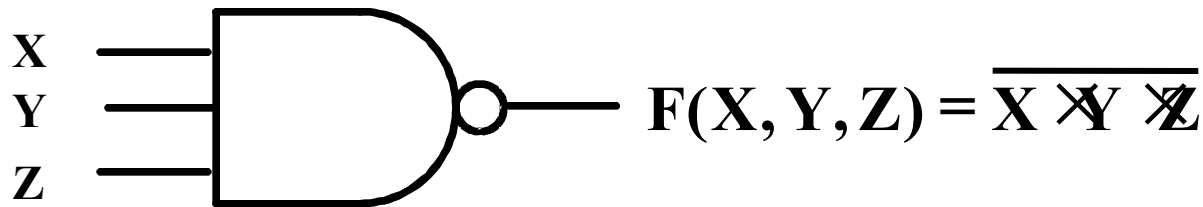


- In terms of Boolean function, a buffer is the same as a connection!
- So why use it?
 - A buffer is an electronic amplifier used to improve circuit voltage levels and increase the speed of circuit operation.

NAND Gate

- The basic NAND gate has the following symbol, illustrated for three inputs:

- AND-Invert (NAND)

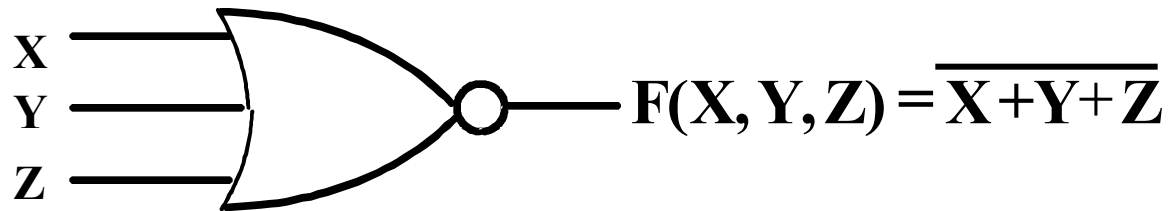


- NAND represents NOT AND, i. e., the AND function with a NOT applied. The symbol shown is an AND-Invert. The small circle (“bubble”) represents the invert function.

NOR Gate

- The basic NOR gate has the following symbol, illustrated for three inputs:

- OR-Invert (NOR)



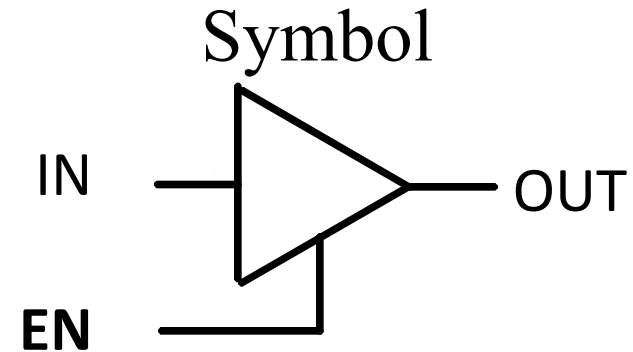
- NOR represents NOT - OR, i. e., the OR function with a NOT applied. The symbol shown is an OR-Invert. The small circle (“bubble”) represents the invert function.

The 3-State Buffer

- ❑ Three-state logic adds a third logic value, Hi-Impedance (Hi-Z), giving three states: 0, 1, and Hi-Z on the outputs.
- ❑ What is a Hi-Z value?
 - The Hi-Z value behaves as an open circuit
 - This means that, looking back into the circuit, the output appears to be disconnected.
 - It is as if a switch between the internal circuitry and the output has been opened.

The 3-State Buffer

- ❑ For the symbol and truth table, **IN** is the data input, and **EN**, the control input.
- ❑ For **EN** = 0, regardless of the value on **IN** (denoted by **X**), the output value is **Hi-Z**.
- ❑ For **EN** = 1, the output value follows the input value.
- ❑ Variations:
 - Data input, **IN**, can be inverted
 - Control input, **EN**, can be inverted by addition of “bubbles” to signals.

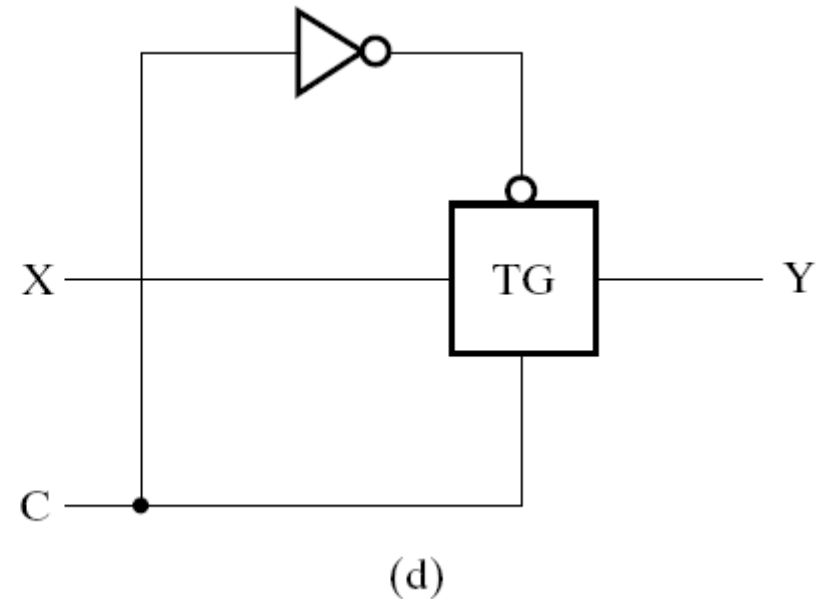
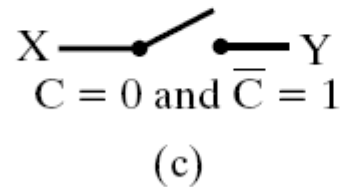
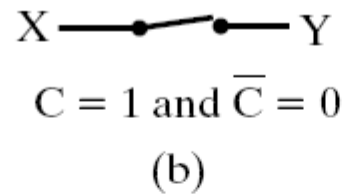
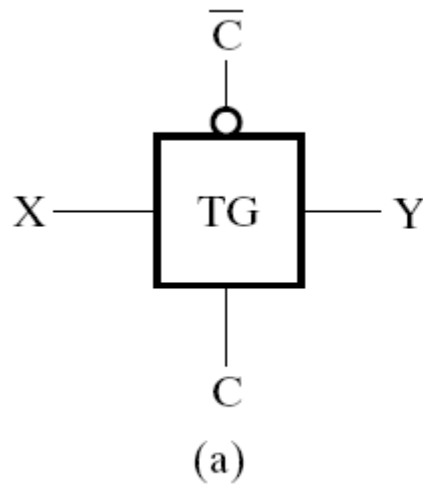


Truth Table

EN	IN	OUT
0	X	Hi-Z
1	0	0
1	1	1

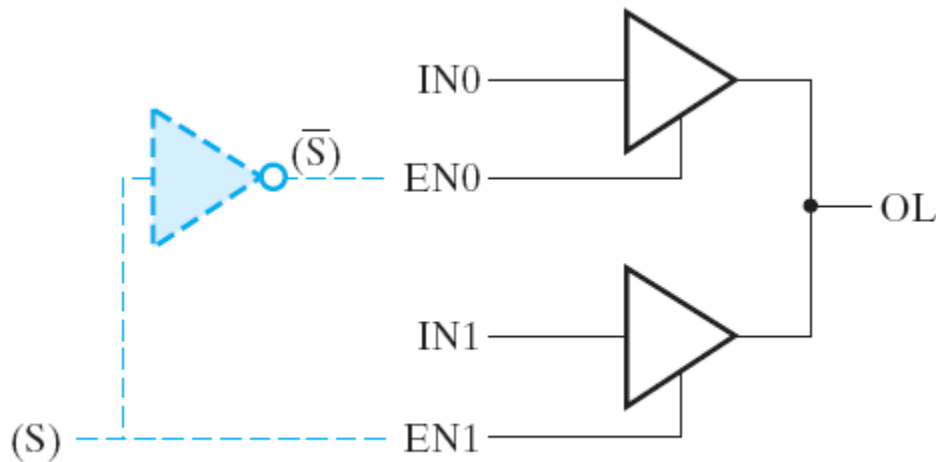
Transmission Gate

□ A transmission gate can be regarded as a switch.



The 3-State Buffer

- Multiplexed output line
 - Formed by 3-state buffers



(a) Logic Diagram

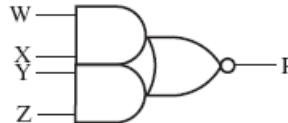
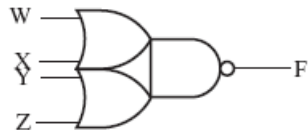
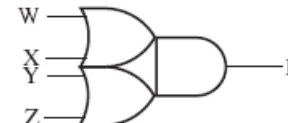
EN1	EN0	IN1	IN0	OL
0	0	X	X	Hi-Z
(S) 0	($\bar{S}$) 1	X	0	0
0	1	X	1	1
1	0	0	X	0
1	0	1	X	1
1	1	0	0	0
1	1	1	1	1
1	1	0	1	
1	1	1	0	

(b) Truth table

- Since $EN0 = \bar{S}$ and $EN1 = S$, one of the two buffer outputs is always Hi-Z plus the first row of the table never occurs.

Complex Gates

□ Symbols & truth tables for complex gates.

Name	Distinctive shape symbol	Algebraic equation	Truth table															
Exclusive-OR (XOR)		$F = X\bar{Y} + \bar{X}Y$ $= X \oplus Y$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	X	Y	F	0	0	0	0	1	1	1	0	1	1	1	0
X	Y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR (XNOR)		$F = XY + \bar{X}\bar{Y}$ $= \overline{X \oplus Y}$	<table><tr><th>X</th><th>Y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	X	Y	F	0	0	1	0	1	0	1	0	0	1	1	1
X	Y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																
AND-OR-INVERT (AOI)		$F = \overline{WX + YZ}$																
OR-AND -INVERT (OAI)		$F = \overline{(W + X)(Y + Z)}$																
AND-OR (AO)		$F = WX + YZ$																
OR-AND (OA)		$F = (W + X)(Y + Z)$																

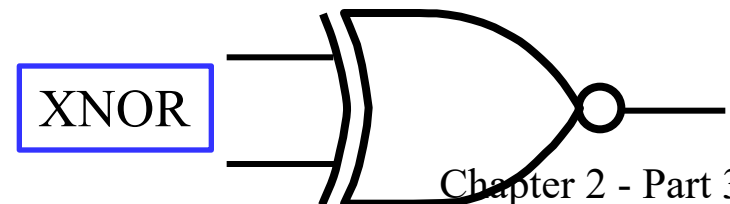
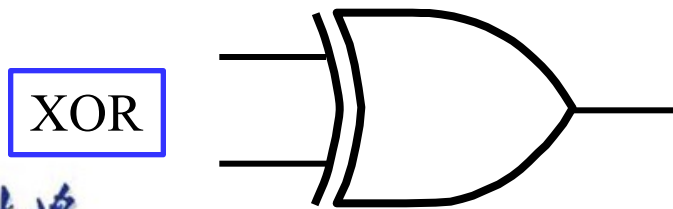
Exclusive OR/ Exclusive NOR

- The *eXclusive OR (XOR)* function is an important Boolean function used extensively in logic circuits.
- The *eXclusive NOR* function is the complement of the XOR function

□ Definitions

- The XOR function is:
$$\mathbf{X \oplus Y = X \bar{Y} + \bar{X} Y}$$
- The eXclusive NOR (XNOR) function, otherwise known as *equivalence* is:
$$\overline{\mathbf{X \oplus Y}} = \mathbf{X Y + \bar{X} \bar{Y}}$$

□ Symbols



Truth Tables for XOR/XNOR

□ Operator Rules: XOR

X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

X	Y	$\overline{(X \oplus Y)}$ or $X \equiv Y$
0	0	1
0	1	0
1	0	0
1	1	1

□ The XOR function means:

X OR Y, but NOT BOTH

□ Why is the XNOR function also known as the *equivalence* function, denoted by the operator $\equiv$?

XOR/XNOR (Continued)

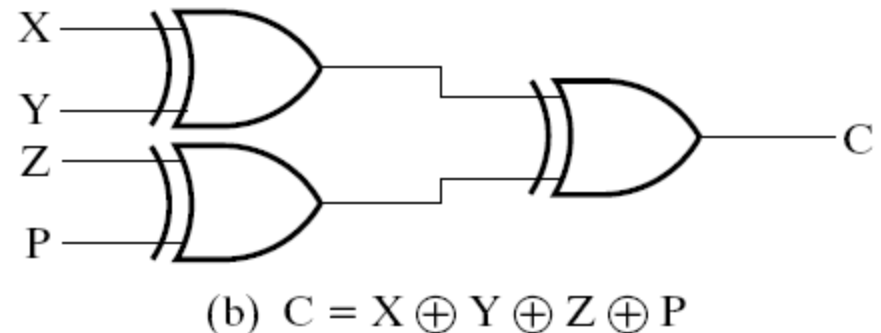
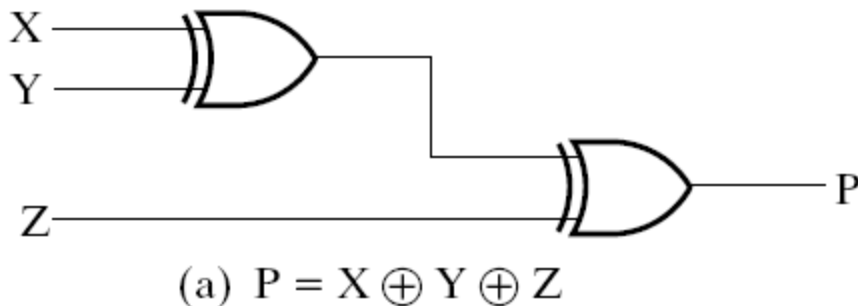
YZ		Y			
		00	01	11	10
X	0		1		1
	1	1		1	

(a) $X \oplus Y \oplus Z$

CD		C			
		00	01	11	10
AB	00		1		1
	01	1		1	
	11		1		1
	10	1		1	

(b) $A \oplus B \oplus C \oplus D$

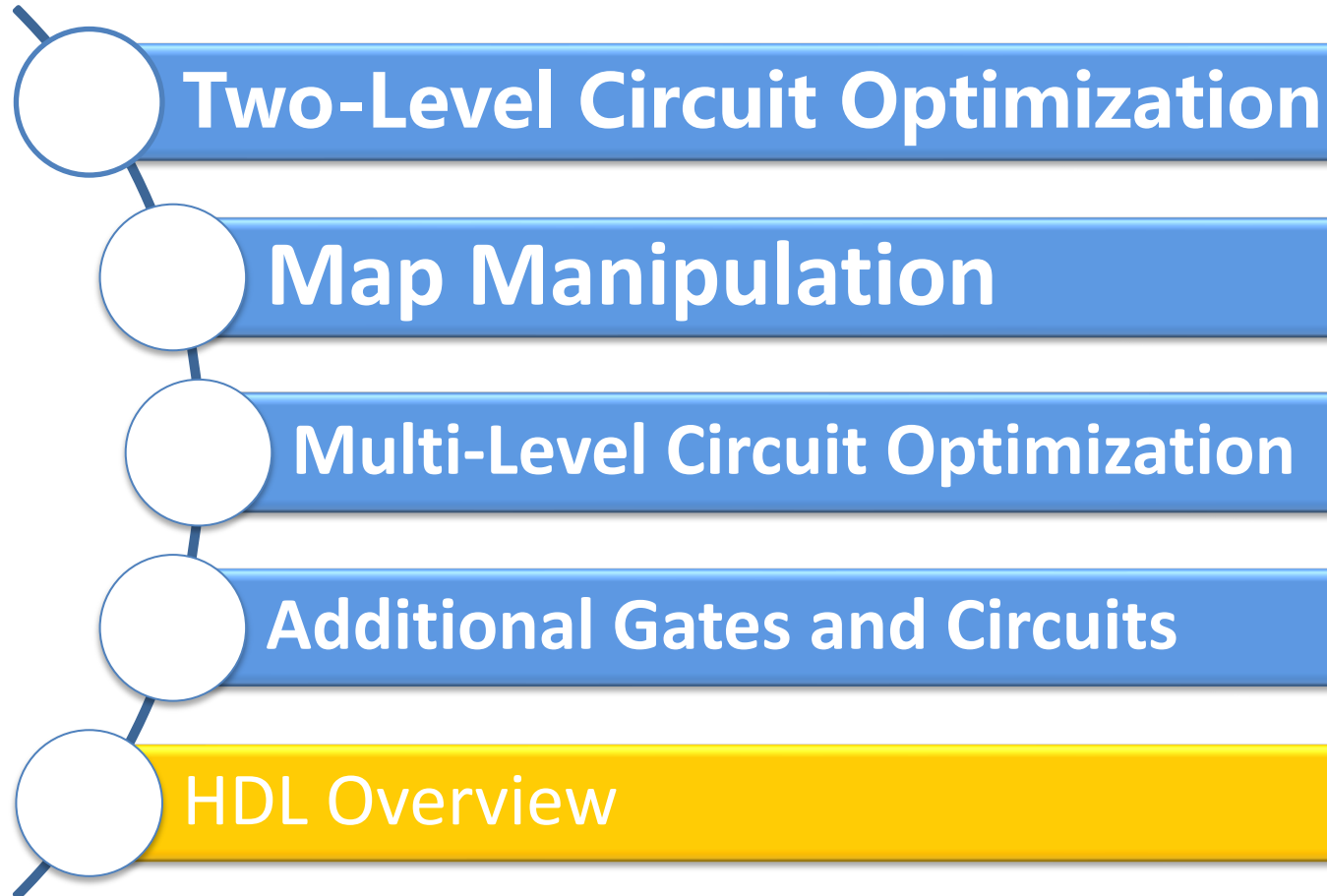
K-maps with odd variables



odd functions with odd inputs

Parity Generators and Checkers

Course Outline

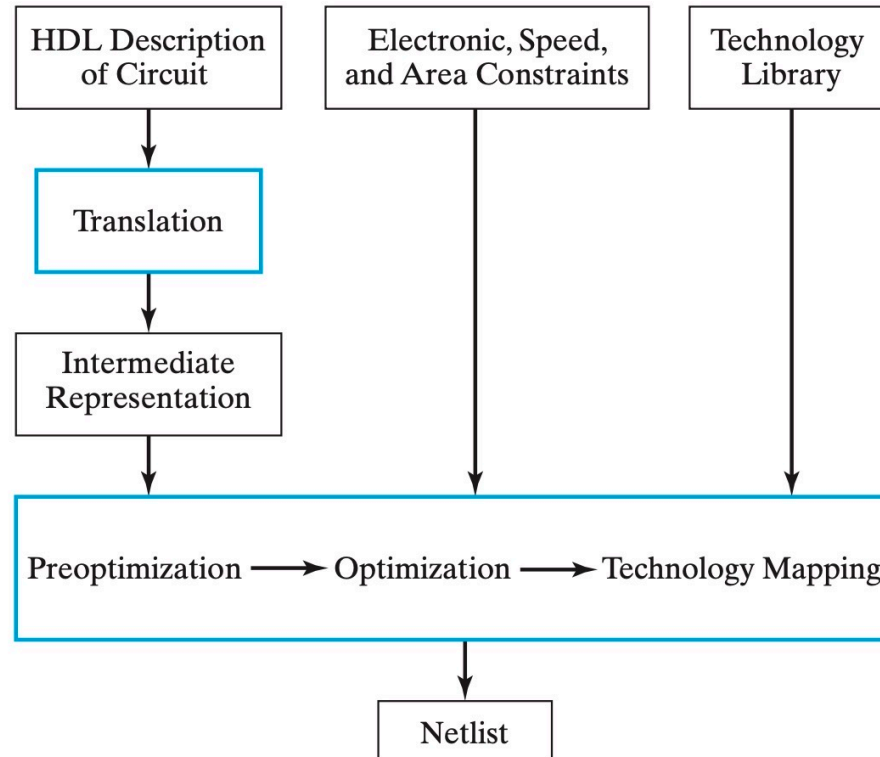


HDL Overview

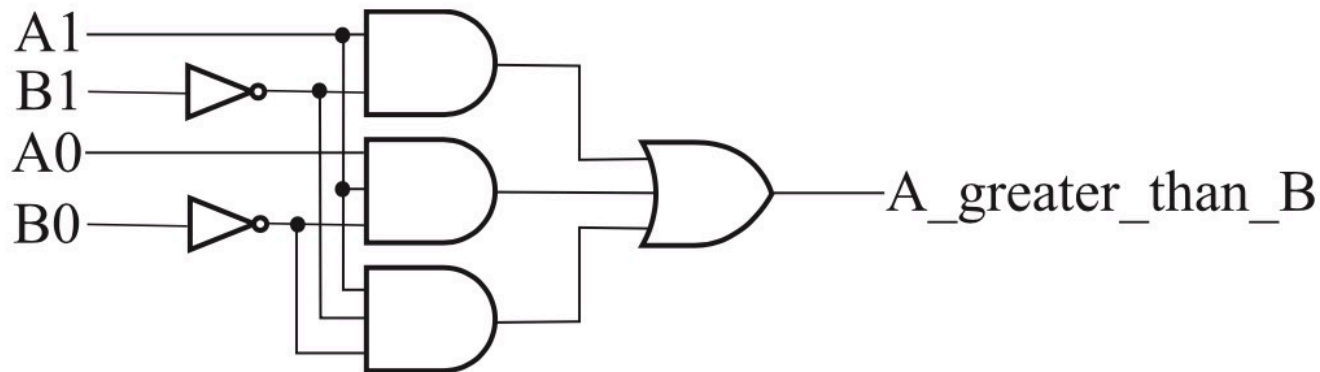
- ❑ **HDL:** hardware description Language(VHDL, Verilog)
- ❑ **RTL:** Register transfer language level
- ❑ **DUT:** Device under test
- ❑ **TestBench:** a collection of hardware and software functions that apply inputs to the DUT and analyze the outputs for correctness.
- ❑ **Logic Simulator:** simulate the HDL to determine the outputs.

Logic Synthesis

- Transform and RTL description of a circuit in an HDL into an optimized netlist representing storage elements and combinational logic



□ Two-bit greater-than comparator circuit



Structural Description

□ Equivalent to the circuit schematic

```
// Two-bit greater-than circuit: Verilog structural model           // 1
// See Figure 2-27 for logic diagram                               // 2
module comparator_greater_than_structural(A, B, A_greater_than_B); // 3
  input [1:0] A, B;                                                 // 4
  output A_greater_than_B;                                          // 5
  wire B0_n, B1_n, and0_out, and1_out, and2_out;                  // 6
  not                                                 // 7
    inv0(B0_n, B[0]), inv1(B1_n, B[1]);                             // 8
  and                                                 // 9
    and0(and0_out, A[1], B1_n),                                     // 10
    and1(and1_out, A[1], A[0], B0_n),                             // 11
    and2(and2_out, A[0], B1_n, B0_n);                             // 12
  or                                                 // 13
    or0(A_greater_than_B, and0_out, and1_out, and2_out);           // 14
endmodule                                                 // 15
```

DataFlow description

- Describe a circuit in terms of function, made up of concurrent assignments or their equivalent.
- Behavioral Description
- Logic level

```
// Two-bit greater-than circuit: Dataflow model // 1
// See Figure 2-27 for logic diagram // 2
module comparator_greater_than_dataflow(A, B, A_greater_than_B); // 3
  input [1:0] A, B; // 4
  output A_greater_than_B; // 5
  wire B1_n, B0_n, and0_out, and1_out, and2_out; // 6
  assign B1_n = ~B[1]; // 7
  assign B0_n = ~B[0]; // 8
  assign and0_out = A[1] & B1_n; // 9
  assign and1_out = A[1] & A[0] & B0_n; // 10
  assign and2_out = A[0] & B1_n & B0_n; // 11
  assign A_greater_than_B = and0_out | and1_out | and2_out; // 12
endmodule // 13
```

Conditional Operator

- Higher level than logic level
- Behavioral Description

```
// Two-bit greater-than circuit: Conditional model           // 1
// See Figure 2-27 for logic diagram                         // 2
module comparator_greater_than_conditional2(A, B, A_greater_than_B); // 3
    input [1:0] A, B;                                         // 4
    output A_greater_than_B;                                   // 5
    assign A_greater_than_B = (A > B)? 1'b1 :                 // 6
        1'b0;                                                 // 7
endmodule                                                  // 8
```



Conditional Operator

```
// Two-bit greater-than circuit: Conditional model           // 1
// See Figure 2-27 for logic diagram                         // 2
module comparator_greater_than_conditional(A, B, A_greater_than_B); // 3
    input [1:0] A, B;                                         // 4
    output A_greater_than_B;                                   // 5
    assign A_greater_than_B = (A == 2'b00)? 1'b0 :           // 6
        (A == 2'b01)? ~(B[1]|B[0]) :                         // 7
        (A == 2'b10)? ~B[1] :                                 // 8
        (A == 2'b11)? ~(B[1]&B[0]) :                         // 9
        1'bx;                                                 // 10
endmodule                                                  // 11
```

```
// Two-bit greater-than circuit: Behavioral model           // 1
// See Figure 2-27 for logic diagram                         // 2
module comparator_greater_than_behavioral(A, B, A_greater_than_B); // 3
    input [1:0] A, B;                                         // 4
    output A_greater_than_B;                                   // 5
    assign A_greater_than_B = A > B;                           // 6
endmodule                                                  // 7
```

TestBench

- A testbench is an HDL model whose purpose is to test another model, often called the Device Under Test (DUT)

```
// Testbench for Verilog two-bit greater-than comparator // 1
module comparator_testbench_verilog(); // 2
    reg [1:0] A, B; // 3
    wire struct_out; // 4
    comparator_greater_than_structural U1(A, B, struct_out); // 5
    initial // 6
    begin // 7
        A = 2'b10; // 8
        B = 2'b00; // 9
        #10; // 10
        B = 2'b01; // 11
        #10; // 12
        B = 2'b10; // 13
        #10; // 14
        B = 2'b11; // 15
    end // 16
endmodule // 17
```



Home Assignment

**2-1a; 2-2a, c; 2-3a, c; 2-6b, d; 2-10a, c;
2-11a, c, d; 2-12b; 2-15; 2-17; 2-19a; 2-
22a; 2-25b; 2-29; 2-30; 2-31;**